WEB ARCHITECTURE & SECURITY

CORS & Static Routing

Complete Technical Blueprint for Cross-Origin Resource Sharing & Static Asset Serving

Document Type: Technical Architecture Guide

Topic: CORS Middleware Security & Static File Routing

Target Audience: Full Stack Developers, Backend Engineers, DevOps

Status: Approved Guide

1. Introduction to CORS (Cross-Origin Resource Sharing)

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that allows or restricts requested resources on a web page to be requested from another domain outside the domain from which the first resource was served.

1.1 Understanding Same-Origin Policy (SOP)

Browsers enforce the Same-Origin Policy (SOP) by default. Two URLs have the same origin if the protocol, port, and host are identical.

> **Origin Comparison Example (Base: http://example.com/app):**
>
> - `http://example.com/api` - **Same Origin** (Matches Protocol, Host, Port)
> - `https://example.com/api` - **Different Origin** (Different Protocol: HTTPS)
> - `http://api.example.com/data` - **Different Origin** (Different Host)
> - `http://example.com:8080/app` - **Different Origin** (Different Port)

1.2 Why is CORS Necessary?

Modern web applications frequently separate frontend clients (e.g., React on port 3000) from API backends (e.g., FastAPI on port 8000). Without explicit CORS headers, the browser blocks client-side JavaScript requests from reading responses.

2. CORS Headers & Preflight Request Mechanics

CORS communication relies on specific HTTP headers exchanged between browser client and server.

2.1 Key CORS Response Headers

Header	Description
`Access-Control-Allow-Origin`	Specifies allowed origins (e.g., `https://app.example.com` or *)
`Access-Control-Allow-Methods`	Lists allowed HTTP methods (e.g., `GET, POST, PUT, DELETE`)
`Access-Control-Allow-Headers`	Specifies acceptable request headers (e.g., `Authorization, Content-Type`)
`Access-Control-Allow-Credentials`	Indicates whether cookies and authorization headers can be included

2.2 Preflight Requests (HTTP OPTIONS)

For non-simple requests (such as POST with JSON or requests with custom headers), the browser sends an HTTP `OPTIONS` request before the actual request to verify server permission.

```
// Preflight Request Sent by Browser
OPTIONS /api/v1/users HTTP/1.1
Host: api.example.com
Origin: http://localhost:3000
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization, content-type

// Server Preflight Response
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: Authorization, Content-Type
```

3. Implementing CORS Middleware

Configuring CORS middleware properly is necessary to enable cross-domain communication while maintaining security.

3.1 FastAPI CORS Middleware Implementation

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Define allowed origin origins
origins = [
    "http://localhost:3000",
    "https://my-smart-project.com"
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/api/data")
async def get_data():
    return {"status": "CORS Enabled Success"}
```

3.2 Express.js CORS Implementation

```
const express = require('express');
const cors = require('cors');
const app = express();

const corsOptions = {
  origin: ['http://localhost:3000', 'https://my-smart-project.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true
};

app.use(cors(corsOptions));
```

4. CORS Security Risks & Production Best Practices

Misconfigured CORS rules introduce serious security vulnerabilities like Cross-Site Request Forgery (CSRF) and unauthorized data exposure.

4.1 Common Misconfigurations

- **Using * wildcard with Credentials:** Browsers reject `allow_origins=["*"]` if `allow_credentials=True`.
- **Reflecting Arbitrary Origins:** Dynamically returning whatever origin header the client sends without validation.
- **Overly Permissive Origins in Production:** Allowing `localhost` origins in live production environments.

4.2 Production CORS Checklist

Setting	Development	Production
Origins	`http://localhost:3000`	Explicit Domain List (e.g., `https://app.com`)
Methods	* (All)	Explicit required methods only
Credentials	Optional	Enable only when using cookies/session tokens

5. Introduction to Static Routing

Static routing refers to serving pre-existing static assets (images, CSS files, JavaScript bundles, HTML documents, PDFs) directly from the server disk filesystem without dynamic server-side rendering or database queries.

5.1 Dynamic Routes vs Static File Routes

Feature	Dynamic Routes	Static Routes
Execution	Runs backend code & queries DB	Reads file directly from disk storage
Response Time	Higher latency (computation)	Ultra-fast (can be cached at CDN)
Content	Generated per request dynamically	Fixed content across requests

5.2 Common Use Cases for Static Routing

- Serving Single Page Application (SPA) frontend build files (React/Vue dist output).
- Serving user uploaded media (profile images, document downloads).
- Exposing API documentation bundles and asset assets.

6. Implementing Static File Routing

Frameworks provide dedicated static mounting mechanisms to bind filesystem directories to endpoint URL paths.

6.1 FastAPI Static Files Mounting

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
import os

app = FastAPI()

# Ensure directory exists
os.makedirs("static", exist_ok=True)

# Mount static directory to /static path
app.mount("/static", StaticFiles(directory="static"), name="static")

# Files inside ./static/images/logo.png
# are now accessible at http://server/static/images/logo.png
```

6.2 Express.js Static Routing

```
const express = require('express');
const path = require('path');
const app = express();

// Serve static files from public directory
app.use('/static', express.static(path.join(__dirname, 'public')));
```

7. Single Page Application (SPA) Routing Fallback

When serving client-rendered frameworks like React or Angular, direct page reloads on sub-routes (e.g., `/dashboard`) result in 404 errors unless HTML fallback routing is configured.

7.1 SPA Fallback Mechanism Architecture

```
Client Request -> URL: /dashboard
       |
       v
Static Router Checks File Exist?
       |
       +---> YES: Return File (e.g., /static/main.js)
       |
       +---> NO:  Return /static/index.html (Client JS Handles Route)
```

7.2 FastAPI Custom SPA Static Route Handler

```
from fastapi.responses import FileResponse

@app.get("/{full_path:path}")
async def serve_spa(full_path: str):
    file_path = os.path.join("static", full_path)
    if os.path.isfile(file_path):
        return FileResponse(file_path)
    # Fallback to index.html for client-side router
    return FileResponse("static/index.html")
```

8. Performance & Caching Optimization

Optimizing static asset delivery reduces server CPU load, bandwidth consumption, and improves end-user page load speeds.

8.1 HTTP Cache-Control Headers

Configuring proper cache headers allows browsers and CDN edge nodes to cache static assets locally.

> **Recommended Cache Headers:**
>
> - **Immutable Assets (Hashed JS/CSS):** `Cache-Control: public, max-age=31536000, immutable`
> - **HTML Files (index.html):** `Cache-Control: no-cache` (Forces revalidation)

8.2 Nginx Reverse Proxy Offloading

```
# Nginx block for ultra-fast static file serving
location /static/ {
    alias /var/www/app/static/;
    expires 30d;
    add_header Cache-Control "public, no-transform";
}
```

9. Best Practice Checklist & Summary Matrix

A combined architecture using secure CORS rules and efficient static routing forms the backbone of modern web applications.

9.1 Architecture Best Practice Checklist

1. Restrict CORS origins explicitly to trusted domain names in production.
2. Ensure Preflight requests (HTTP OPTIONS) respond with appropriate status codes and cache limits.
3. Mount static asset directories using dedicated prefixes (e.g., `/static` or `/assets`).
4. Implement index.html fallback handling when serving client-side SPAs.
5. Offload heavy static asset serving to CDNs or Nginx in high-traffic production setups.

9.2 Cors vs Static Routing Summary

Dimension	CORS Security	Static Routing
Domain Level	Handles cross-origin request permissions	Serves local files directly from server disk
Primary Layer	Middleware / HTTP Response Headers	File system mount / Reverse Proxy
Key Risk	CSRF / Data exposure misconfiguration	Directory traversal vulnerability

"Proper CORS configuration secures API boundaries, while efficient static routing delivers frontend assets at peak performance."